

VICTORIA SUGAR LIMITED

Corporate Website & Admin Panel

Software Documentation

Technical Reference — Architecture, Codebase, Database, Environment Variables & Credentials

Prepared: 22 July 2026

Sites covered: victoriasugar.ug • admin.victoriasugar.ug

Repository: https://github.com/IsmailSebz/vsl_admin_website / <https://github.com/IsmailSebz/victoriasugarug>

CONFIDENTIAL — CONTAINS LIVE CREDENTIALS

This document reproduces the contents of the project's .env file and hardcoded API keys found in the source code, including a Supabase service-role key, database password, and Cloudflare R2 access keys. Treat this file with the same care as a password vault. Do not email it, upload it to a public drive, or commit it to a public repository. See Section 11 (Security Findings) for urgent recommendations, including the fact that the service-role key is currently exposed to every website visitor.

1. Executive Summary

Victoria Sugar Limited's web presence consists of a single static website codebase that serves two audiences: a public marketing site (victoriasugar.ug) and a password-protected content-management panel (admin.victoriasugar.ug, served from the `/admin` folder of the same codebase). There is no separate "admin website" project on disk — the admin panel is a subset of pages inside the same repository, distinguished only by folder path and by requiring a Supabase-authenticated session.

The site is built as plain HTML, CSS and vanilla JavaScript with no build step, no bundler, and no `package.json`. Styling is done with the Tailwind Play CDN plus hand-written CSS files, and data is stored in a Supabase (hosted Postgres) project. Images and video are stored separately in a Cloudflare R2 bucket and served from a custom subdomain. The site is hosted on Cloudflare Pages.

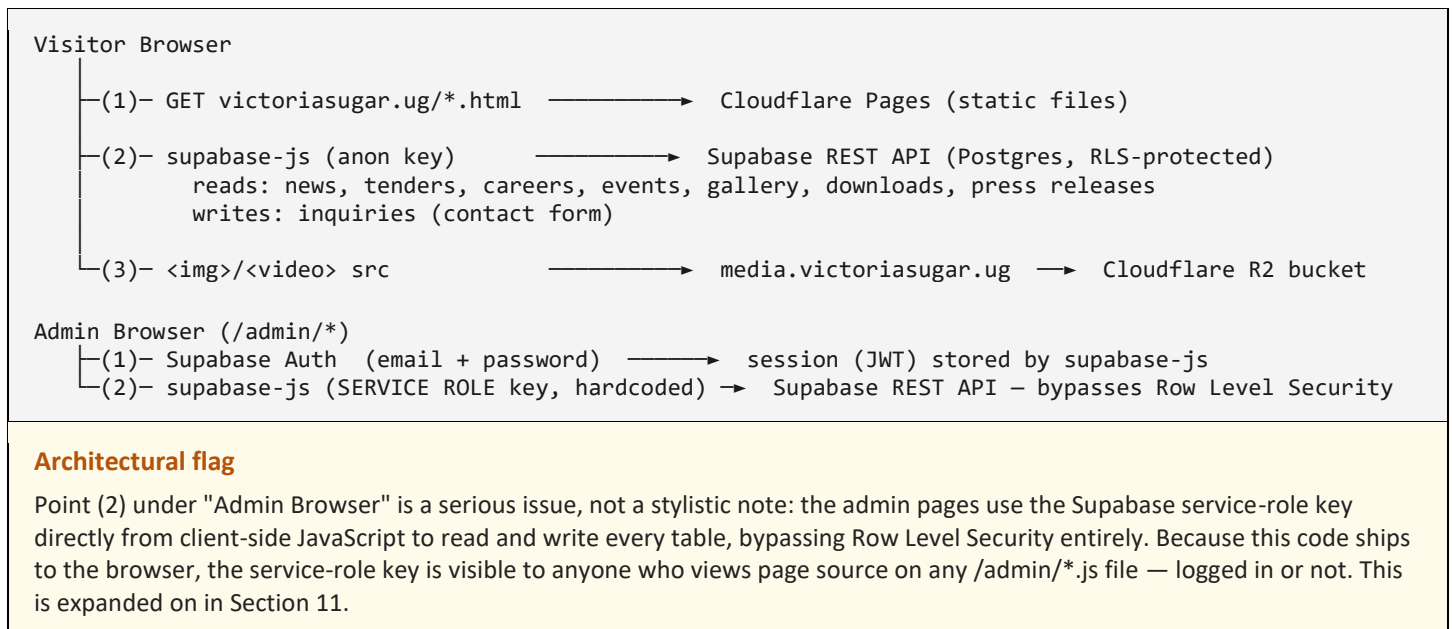
This document explains the architecture, walks through every functional area of the public site and the admin panel with code excerpts, documents the full Supabase database schema as it exists today (pulled live from the project), reproduces every environment variable and secret found in the project's `.env` file and source code, and flags a set of security and data-consistency issues that should be addressed.

2. System Architecture

The system has three moving parts:

- Frontend — static HTML/CSS/JS deployed to Cloudflare Pages. No server-side rendering, no Node.js runtime in production. Every page is a plain `.html` file; JavaScript modules under `/js` run entirely in the visitor's browser.
- Backend-as-a-service — Supabase, providing Postgres (data), Supabase Auth (the one admin login), and the auto-generated REST API used by the Supabase JS client library.
- Media storage — Cloudflare R2 (S3-compatible object storage), holding images/video for news, gallery, downloads, etc., served through the custom domain media.victoriasugar.ug rather than through Supabase Storage.

2.1 Request Flow



3. Technology Stack

Layer	Technology	Notes
Markup / structure	Static HTML5	One .html file per page; no templating engine
Styling	Tailwind CSS (Play CDN) + custom CSS	cdn.tailwindcss.com loaded at runtime; per-page CSS files in /css/pages/
Client scripting	Vanilla JavaScript (ES6+)	No framework (no React/Vue); one script per page in /js/pages/
Database	Supabase (Postgres 17.6, region eu-west-2)	Project ref ffsddbttgobxqlrncrm
Auth	Supabase Auth (email + password)	Single admin account model
Object storage	Cloudflare R2 (S3-compatible)	Bucket vsl-media, served via media.victoriasugar.ug
Hosting / CDN	Cloudflare Pages	Static file hosting, _headers file for security headers
Fonts	Google Fonts (Inter, and Georgia as system serif)	Loaded via <link> tag
Transactional email	Resend (configured, not yet wired up)	RESEND_API_KEY is still a placeholder
Messaging	WhatsApp Click-to-Chat	wa.me link, no API integration
Version control	Git	12 commits at time of writing, most recent: "fixed sending enquiries"

4. Repository Structure

The entire project — public site and admin panel — lives in one folder. Relevant structure (media/binary assets omitted):

<pre> vsl website/ ├── .env ├── .gitignore ├── _headers ├── index.html, about.html, contact.html, faqs.html, sitemap.html, ... ├── products/ ├── news/ ├── press-room/, events/, gallery/, downloads/, careers/, tenders/, sustainability/ ├── admin/ │ ├── login.html │ ├── index.html │ ├── news.html, gallery.html, careers.html, tenders.html, events.html, │ └── press-room.html, downloads.html, inquiries.html, newsletter.html, settings.html ├── css/ │ ├── main.css, admin.css │ └── pages/ ├── js/ │ ├── supabase-client.js │ ├── nav.js, footer.js │ ├── inquiry-modal.js │ ├── common-init.js │ └── pages/ └── resources/, logo.jpg, hero-bg.jpeg, ... </pre>	← secrets (Section 10) – gitignored ← Cloudflare Pages security headers (index, sugar, alcohol, ethanol, electricity) (index, article) ← the "admin website" – password protected panel (dashboard) one stylesheet per page (public + admin) shared Supabase client (public + service-role) shared navbar / footer, injected on every page shared "Send an Inquiry" popup + form handler one-line bootstrap: <code>initNav(); initFooter(); initInquiryModal();</code> one script per page (public + admin), same naming as <code>/admin/*.html</code> (images used by the marketing pages)
--	--

5. Public Website

The public site is the marketing/corporate presence at victoriasugar.ug. It is entirely read-only against the database except for two write paths: the contact/inquiry form and (schema-only, see Section 12) a newsletter subscription table.

5.1 Site Map

Page	Path	Data source
Home	/	static content + highlights pulled via js/pages/home.js
About	/about.html	static
Products (index, sugar, alcohol, ethanol, electricity)	/products/*	static
Bulk Orders / Export	/bulk-orders.html, /export.html	static
News & Media	/news/	table: news
News Article	/news/article.html?id=...	table: news
Press Room	/press-room/	table: press_releases
Gallery	/gallery/	tables: gallery_albums, gallery_items
Events	/events/	table: events
Downloads	/downloads/	table: downloads
Careers	/careers/	table: careers
Tenders	/tenders/	table: tenders
Contact	/contact.html	writes to table: inquiries
Sustainability (4 sub-pages)	/sustainability/*	static
FAQs, Sitemap, Video Tour	/faqs.html, /sitemap.html, /video-tour.html	static

5.2 Shared Components

Every public page loads the same three scripts via a one-line bootstrap in common-init.js:

```
initNav();initFooter();initInquiryModal();
```

nav.js and footer.js render the navbar and footer entirely in JavaScript (they build an HTML string and inject it into #navbar / #footer placeholder divs), so the same markup and links are guaranteed identical on every page. The navigation config is a single array:

```
const NAV_LINKS = [
  { label: 'Home', href: '/' },
  { label: 'About', href: '/about.html' },
  { label: 'Our Products', href: '/products/', children: [
    { label: 'All Products', href: '/products/' },
    { label: 'Sugar', href: '/products/sugar.html' },
    { label: 'Extra Natural Alcohol', href: '/products/alcohol.html' },
    { label: 'Ethanol', href: '/products/ethanol.html' },
    { label: 'Electricity', href: '/products/electricity.html' } ] },
  { label: 'Media', href: '/news/', children: [ ... News, Gallery, Downloads, Events ... ] },
  { label: 'Careers', href: '/careers/' },
  { label: 'Contact', href: '/contact.html' },
]
```

The WhatsApp click-to-chat button (present in both the navbar and footer) is generated from a single constant:

```
const WA = '256749293610'
const WA_URL =
`https://wa.me/${WA}?text=Hello%20Victoria%20Sugar%20Limited%2C%20I%20would%20like%20to%20make%20an%20inquiry.`
```

5.3 Reading Data From Supabase (public pages)

Public pages use the shared, unprivileged client defined in `js/supabase-client.js`:

```
const SUPABASE_URL = 'https://ffsddbbtgoxbqlrncrm.supabase.co'
const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIs... ' // JWT, role: anon
const { createClient } = supabase
const db = createClient(SUPABASE_URL, SUPABASE_ANON_KEY) // respects Row Level Security
```

Typical read pattern, e.g. `Tenders (js/pages/tenders.js)` splits open vs. closed tenders by comparing the deadline column to "now":

```
const now = new Date().toISOString()
const { data: openData } = await db.from('tenders').select('*')
  .gte('deadline', now).order('deadline', {ascending: true})
const { data: closedData } = await db.from('tenders').select('*')
  .lt('deadline', now).order('deadline', {ascending: false}).limit(5)
```

`Gallery (js/pages/gallery.js)` is two-table: albums, then items filtered by album:

```
const { data: albums } = await db.from('gallery_albums').select('*')
  .eq('is_published', true).order('sort_order', {ascending: true})
let q = db.from('gallery_items').select('*').order('sort_order', {ascending: true})
```

5.4 Contact / Inquiry Flow

The "Send an Inquiry" modal (`js/inquiry-modal.js`) is injected on every page and is also mirrored as a full page at `/contact.html (js/pages/contact.js)`. Both submit directly to the inquiries table using the anon key — no email is actually sent server-side; the admin sees new inquiries by logging into the panel. Submit handler:

```
const data = { name: fd.get('name'), email: fd.get('email'), phone: fd.get('phone'),
  subject: fd.get('subject'), message: fd.get('message') }
const { error } = await db.from('inquiries').insert([data])
// on success: shows "Thank you! We have received your inquiry..." and auto-closes after 3s
// on error: falls back to suggesting WhatsApp
```

6. Admin Panel (Content Management System)

The admin panel lives at `/admin/` (published as `admin.victoriasugar.ug`) and is the only way to create, edit, publish or delete content — there is no separate authoring tool. It is a set of 10 HTML pages, each paired with a same-named script in `js/pages/admin-*.js`, all sharing the same login gate, sidebar, and Supabase client setup.

6.1 Authentication

Login (`admin/login.html` + `js/pages/admin-login.js`) uses Supabase Auth's password grant. If a session already exists, the user is redirected straight to the dashboard:

```
const db = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
(async () => {
  const { data: { session } } = await db.auth.getSession();
  if (session) window.location.href = "/admin/";
})();

loginForm.addEventListener("submit", async e => {
  e.preventDefault();
  const { error } = await db.auth.signInWithPassword({
    email: emailInput.value, password: passwordInput.value
  });
  if (error) showError(error.message); else window.location.href = "/admin/";
});
```

Every other admin page repeats the same guard clause at the top of its script (copy-pasted, not shared as a module):

```
async function checkAuth() {
  const { data: { session } } = await db.auth.getSession();
  if (!session) { window.location.href = '/admin/login.html'; return null; }
  return session;
}
// logout:
logoutBtn.addEventListener("click", async () => { await db.auth.signOut(); window.location.href =
"/admin/login.html"; });
```

There is exactly one privileged account model: whoever Supabase Auth recognises as a user can sign in and gets full access — there are no per-user roles or permission tiers inside the app itself (an `admin_profile` table exists 1:1 with `auth.users` but is only used to store a display name/avatar, not a role).

6.2 Dashboard

`admin/index.html` (`js/pages/admin-dashboard.js`) shows four live counters and the five most recent inquiries, fetched in parallel:

```
const [newsRes, tendersRes, inqRes, subsRes] = await Promise.all([
  dbAdmin.from('news_articles').select('id', { count: 'exact', head: true }).eq('published', true),
  dbAdmin.from('tenders').select('id', { count: 'exact', head: true }).gte('deadline', now),
  dbAdmin.from('inquiries').select('id', { count: 'exact', head: true }).eq('is_read', false),
  dbAdmin.from('newsletter_subscribers').select('id', { count: 'exact', head: true }),
]);
```

Bug

The dashboard queries table `news_articles` and columns `published/published_date`, but the real table (Section 7) is named `news` with columns `is_published / published_date`. This counter will always error/return null — see Section 12, item 1.

6.3 Content Modules

Each module follows the same list → modal-form → insert/update/delete pattern against a dedicated table, always via the service-role client (`dbAdmin`):

Admin page	Script	Table used	Key fields managed
Dashboard	<code>admin-dashboard.js</code>	<code>news_articles*</code> , <code>tenders</code> , <code>inquiries</code> , <code>newsletter_subscribers</code>	read-only counters + recent inquiries
News	<code>admin-news.js</code>	<code>news</code>	<code>title</code> , <code>is_published</code> , <code>published_date</code>
Gallery	<code>admin-gallery.js</code>	<code>gallery_albums</code> , <code>gallery_items</code>	albums (<code>title</code> , <code>sort_order</code> , <code>is_published</code>), items (<code>type</code> , <code>url</code> , <code>thumbnail_url</code>)
Careers	<code>admin-careers.js</code>	<code>careers</code>	<code>title</code> , <code>department</code> , <code>location</code> , <code>deadline</code> , <code>is_active</code>
Tenders	<code>admin-tenders.js</code>	<code>tenders</code>	<code>title</code> , <code>reference*</code> , <code>deadline</code>
Events	<code>admin-events.js</code>	<code>events</code>	<code>title</code> , <code>location</code> , <code>event_date</code>
Press Room	<code>admin-press-room.js</code>	<code>press_releases</code>	<code>title</code> , <code>category*</code> , <code>published*</code> , <code>published_at*</code>
Downloads	<code>admin-downloads.js</code>	<code>downloads</code>	<code>title</code> , <code>category</code> , <code>file_url</code> , <code>file_type</code>
Inquiries	<code>admin-inquiries.js</code>	<code>inquiries</code>	list/filter by <code>is_read</code> , view full message
Newsletter	<code>admin-newsletter.js</code>	<code>newsletter_subscribers</code>	list + remove; count of active subscribers
Settings	<code>admin-settings.js</code>	(Supabase Auth) <code>db.auth.updateUser</code>	change own password, view account email / last sign-in

* Fields/tables marked with an asterisk do not match the live database schema in Section 7 — see Section 12 for the full list of these mismatches.

6.3.1 CRUD example (Tenders)

This is the representative pattern used by every module (`Careers`, `Events`, `Downloads`, `Press Room`, `Tenders` all use this exact shape):

```
// Load list
const {data} = await dbAdmin.from('tenders').select('id,title,reference,deadline')
  .order('deadline',{ascending:false});

// Load one record into the edit modal
async function editItem(id){
  const {data} = await dbAdmin.from("tenders").select("*").eq("id",id).single();
  openModal(data);
}

// Delete (with confirm dialog)
async function deleteItem(id){
  if(!confirm("Delete this record?")) return;
  await dbAdmin.from("tenders").delete().eq("id",id);
  loadItems();
}

// Create or update, decided by whether editId is set
const {error} = editId
```

```
? await dbAdmin.from("tenders").update(payload).eq("id",editId)
: await dbAdmin.from("tenders").insert([payload]);
```

6.4 Media Uploads

Admin forms for News, Gallery, Downloads etc. take a plain `file_url` / `cover_image_url` / `thumbnail_url` text field rather than an in-panel file uploader — there is no code path in the repository that uploads a file to Cloudflare R2. In practice, whoever manages content must upload the image/video to the R2 bucket separately (e.g. via the Cloudflare dashboard, `rclone`, or the S3 API using the R2 keys in Section 10) and then paste the resulting `https://media.victoriasugar.ug/...` URL into the admin form.

7. Database Schema (Supabase / Postgres)

Live schema, pulled directly from the Supabase project (ref ffsdabbtgoxbqlrncrm, Postgres 17.6, region eu-west-2, host db.ffsdabbtgoxbqlrncrm.supabase.co). Row Level Security (RLS) is enabled on every table.

public.news (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
body_html	text	—	rich HTML article body
cover_image_url	text	null	R2 media URL
excerpt	text	null	
published_date	date	CURRENT_DATE	
is_published	boolean	true	
category	text	null	
created_at / updated_at	timestampz	now()	

public.gallery_albums (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
description	text	null	
cover_image_url	text	null	
sort_order	integer	0	
is_published	boolean	true	
created_at / updated_at	timestampz	now()	

public.gallery_items (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
album_id	uuid	—	FK → gallery_albums.id
type	text	—	check: 'image' or 'video'
url	text	—	R2 media URL
thumbnail_url	text	null	

Column	Type	Default	Notes
title / description	text	null	
file_name / file_size	text / bigint	null	
duration_seconds	integer	null	for video items
sort_order	integer	0	
created_at	timestampz	now()	

Foreign key: *gallery_items.album_id* → *gallery_albums.id*

public.tenders (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
reference_number	text	null	admin UI reads this as "reference" — mismatch
description	text	null	
file_url	text	null	tender document, R2 or external URL
deadline	date	null	
is_active	boolean	true	
created_at / updated_at	timestampz	now()	

public.careers (rows: 1, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
department	text	null	
location	text	'Masaka, Uganda'	
description	text	—	
requirements	text	null	
how_to_apply	text	null	
deadline	date	null	
is_active	boolean	true	
created_at / updated_at	timestampz	now()	

public.press_releases (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
body_html	text	—	
published_date	date	CURRENT_DATE	admin UI reads this as "published_at" — mismatch
is_published	boolean	true	admin UI reads this as "published" — mismatch
created_at / updated_at	timestampz	now()	

public.events (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
description	text	null	
event_date	date	—	
event_end_date	date	null	
location	text	null	
image_url	text	null	
is_active	boolean	true	
created_at / updated_at	timestampz	now()	

public.downloads (rows: 1, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
title	text	—	
category	text	null	
file_url	text	—	R2 media/document URL
file_size	bigint	null	
file_type	text	null	e.g. pdf, docx
created_at	timestampz	now()	

public.inquiries (rows: 1, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key

Column	Type	Default	Notes
name / email / phone	text	—/—/null	phone optional
subject	text	null	
message	text	—	
inquiry_type	text	'general'	
is_read	boolean	false	
replied	boolean	false	
replied_at	timestampz	null	
reply_message	text	null	no UI currently writes a reply — see Section 12
created_at	timestampz	now()	

public.newsletter_subscribers (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
email	text	—	unique
is_active	boolean	true	
created_at	timestampz	now()	admin UI reads this as "subscribed_at" — mismatch

public.site_settings (rows: 0, RLS: enabled)

Column	Type	Default	Notes
key	text	—	primary key
value	text	null	
description	text	null	
updated_at	timestampz	now()	

Key/value settings table — currently empty; no admin page reads or writes it yet.

public.admin_audit_log (rows: 0, RLS: enabled)

Column	Type	Default	Notes
id	uuid	uuid_generate_v4()	primary key
action	text	—	check: INSERT / UPDATE / DELETE
table_name / record_id / record_title	text	—	
old_data / new_data	jsonb	null	
performed_at	timestampz	now()	

Designed as an audit trail for admin changes — currently empty; nothing in the codebase writes to it yet.

public.admin_profile (rows: 1, RLS: enabled)

Column	Type	Default	Notes
id	uuid	—	PK, FK → auth.users.id
display_name	text	'Administrator'	
avatar_url	text	null	
created_at / updated_at	timestampz	now()	

Foreign key: admin_profile.id → auth.users.id

8. Environment Variables & Secrets

Handle as a credential vault

Everything in this section is a live, working secret as of 22 July 2026, taken from the project's own .env file and from hardcoded values in the JavaScript source. Rotate any of these immediately if this document is ever shared beyond the people who already have access to the Supabase and Cloudflare dashboards.

8.1 Project .env file (verbatim)

Location: vsl website/.env — listed in .gitignore, so it is not committed to Git. Full contents:

```
# =====
# Victoria Sugar Limited – Environment Variables
# =====
# NEVER commit this file to GitHub. It is in .gitignore.
# Add these same variables in Cloudflare Pages → Settings → Environment Variables
# =====

# — Supabase —————
NEXT_PUBLIC_SUPABASE_URL=https://ffsddbttgoxbqlrncrm.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImZmc2RkYmJ0Z294YnF3IiwiaWF0Ij0iMTY2MjY2OTYyLjA9In0=
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImZmc2RkYmJ0Z294YnF3IiwiaWF0Ij0iMTY2MjY2OTYyLjA9In0=
3JA

# — Cloudflare R2 —————
R2_ACCOUNT_ID=4fb1ca4024d02baee6d3b7836495889c
R2_ACCESS_KEY_ID=53b0a94bc2585c8577c4e3273d8f279e
R2_SECRET_ACCESS_KEY=d966077b4cccb70c6973ea0771389869e8fc8481c2e30c53cd98e15db45cb33e
R2_BUCKET_NAME=vsl-media
R2_ENDPOINT=https://4fb1ca4024d02baee6d3b7836495889c.r2.cloudflarestorage.com
NEXT_PUBLIC_MEDIA_URL=https://media.victoriasugar.ug

# — Email (Resend) – add key when ready —————
RESEND_API_KEY=re_placeholder_add_your_key_here
CONTACT_EMAIL=info@victoriasugar.com

# — App URLs —————
NEXT_PUBLIC_SITE_URL=https://victoriasugar.ug
NEXT_PUBLIC_ADMIN_URL=https://admin.victoriasugar.ug
```

Important — these variables are not currently used

Because the site has no build step (no Next.js, no bundler, no server), nothing in the deployed site actually reads process.env or these NEXT_PUBLIC_* variables. The live site instead has the Supabase URL and both the anon key and the service-role key hardcoded directly, in plaintext, inside every one of the 13 JS files. Changing a value there changes anything in production — the corresponding hardcoded string in the JS file must also be changed.

8.2 Hardcoded credentials in source (client-side JavaScript)

Found identically repeated in: js/supabase-client.js, js/pages/admin-login.js, admin-dashboard.js, admin-news.js, admin-gallery.js, admin-careers.js, admin-tenders.js, admin-events.js, admin-press-room.js, admin-downloads.js, admin-inquiries.js, admin-newsletter.js, admin-settings.js (13 files total):

These were shared as setup notes and are not yet formalised as environment variables anywhere, but are live and in active use:

Credential	Value	Where it's used
Supabase DB password	e3LE5cPyNNr9Mwwy	Direct Postgres connection (psql / migrations) — not used by the website itself
Supabase project name	sebzismail Project	Supabase dashboard label
Supabase project ID / ref	ffsddbbtgoxbqlrnnvcrm	Used to build every Supabase URL
Supabase publishable key	sb_publishable_BQvGZjG29groWD4mBMKpDg_9GDfwq-O	Newer-style Supabase publishable key; not currently referenced in code (anon JWT is used instead)
Cloudflare account ID	4fb1ca4024d02baee6d3b7836495889c	R2 dashboard / API
R2 S3 API endpoint (bucket)	https://4fb1ca4024d02baee6d3b7836495889c.r2.cloudflarestorage.com/vsl-media	S3-compatible API access to the media bucket
R2 public dev URL	https://pub-d1720a72844c4d1ba0acd13915c425ea.r2.dev	Fallback public URL for the bucket before the custom domain was attached
Media image base URL	https://media.victoriasugar ug/images/	Used as prefix for cover_image_url etc.
Media video base URL	https://media.victoriasugar ug/videos/	Used as prefix for gallery_items.url (video type)

Cloudflare Pages → the project → Settings → Environment Variables is where `NEXT_PUBLIC_*`, `R2_*`, `RESEND_API_KEY` etc. should be entered if/when the site migrates to a build step that reads them. Until then, any change to a Supabase key or URL must be edited by hand in all 13 JavaScript files listed in 8.2 (a good first refactor — see Section 11 — is to centralise these into `supabase-client.js` only and have every page script include that one file).

9. Third-Party Services & Integrations

9.1 Supabase

Provides Postgres, Auth and the auto-generated REST API. Project "sebzismail Project", ref ffsddbttgoxbqlrncrm, Postgres 17.6.1, region eu-west-2 (London), organisation inxbedhmloswlgylpbw. Created 28 June 2026.

9.2 Cloudflare R2

S3-compatible object storage bucket vsl-media holds all images and video referenced by the news, gallery, downloads, careers and events tables. Served publicly through the custom domain media.victoriasugar.ug (with images/ and videos/ path prefixes) rather than the default r2.dev URL, which is kept as a fallback.

9.3 Cloudflare Pages

Static hosting for the whole site, including /admin. A single _headers file controls response headers for the deployment:

```
/*
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Referrer-Policy: strict-origin-when-cross-origin

/admin/*
  X-Robots-Tag: noindex
```

The /admin/* rule keeps the admin panel out of search engine indexes; it is not an access-control mechanism (the panel is still reachable by anyone who knows or guesses the URL — the only gate is the Supabase Auth login screen).

9.4 Resend (email)

Configured in .env with a placeholder key (RESEND_API_KEY=re_placeholder_add_your_key_here) and a CONTACT_EMAIL. Not yet wired into any code path — inquiry form submissions currently only write a database row, no email notification is sent to staff.

9.5 WhatsApp

No API integration — implemented purely as wa.me deep links ("click-to-chat") pointing at +256 749 293610, used on the navbar, footer, and as an error-fallback if the inquiry form fails to submit.

9.6 CDNs

Tailwind CSS is loaded at runtime from cdn.tailwindcss.com (the "Play CDN", intended by Tailwind for prototyping — see Section 11) and Google Fonts serves the Inter typeface.

10. Deployment

There is no CI/CD pipeline defined in the repository and no package.json — deployment is Cloudflare Pages' native "connect a Git repo and serve the files as-is" mode, or a direct upload of the folder. There is no build command to run; every file is served exactly as committed.

- Public site: victoriasugar.ug — Cloudflare Pages project root = repository root.

- Admin panel: admin.victoriasugar.ug — same deployment, mapped to the /admin subfolder via a custom domain/route (or served at victoriasugar.ug/admin/ and aliased).
- To update environment-driven values (once a build step exists), set them under Cloudflare Pages → Settings → Environment Variables using the names in Section 8.1.
- Git history (12 commits) shows the project was migrated from an earlier version to "static web" and iteratively cleaned up (CSS/JS separated into files, resources fixed, download loading fixed, inquiry sending fixed) — see Section 13.

11. Security Findings & Recommendations

Critical — Service-role key is public

The Supabase service-role key (which bypasses Row Level Security entirely — full read/write/delete on every table) is hardcoded in plaintext inside 12+ JavaScript files that are served, unminified, to every visitor's browser at URLs like `victoriasugar.ug/js/pages/admin-news.js`. Anyone can open browser dev tools or "view source", copy that key, and read or modify every row in every table directly against the Supabase REST API — with no need to ever log in. This is the single highest-priority fix.

Recommended fix (in order of effort): (1) rotate the service-role key in the Supabase dashboard immediately, since it is already exposed in the current deployment; (2) stop shipping any service-role key to the browser — instead, write tight RLS policies so the admin's own authenticated session (using only the anon key) is allowed to read/write the tables it needs, scoped to authenticated users; (3) for anything that genuinely needs to bypass RLS (e.g. bulk admin operations), move that logic to a Cloudflare Pages Function / Supabase Edge Function that holds the service-role key server-side only and is called from the browser over HTTPS with the user's session token.

- Duplicated secrets: the same anon/service-role keys are copy-pasted into 13 separate files. A single edit point (import from `supabase-client.js` everywhere) would reduce the chance of an inconsistent or stale key shipping in one page.
- No RLS-scoped admin role: `admin_profile` links 1:1 to `auth.users` but carries no `role/permission` field — anyone who obtains any valid Supabase Auth session for this project is a full admin. If more than one admin account is ever created, this should be revisited.
- Tailwind Play CDN in production: `cdn.tailwindcss.com` is explicitly documented by Tailwind as unsuitable for production (larger payload, runtime compilation, no purging). Fine for now given the small page count, but worth migrating to a compiled Tailwind build before the site grows.
- `.env` is gitignored correctly, which is good — but confirm the same secrets were never committed in an earlier commit (run a history scan, e.g. `git log -p -- .env`, and rotate keys if they ever appear).
- No rate limiting or CAPTCHA on the public inquiries insert — since the anon key can insert freely, the contact form (and the underlying anon key) could be scripted for spam. Consider a Cloudflare Turnstile check or a serverless function in front of the insert.
- `RESEND_API_KEY` is a placeholder, so no confirmation email or staff alert currently fires when someone submits an inquiry or subscribes — new inquiries are only visible by manually checking the admin dashboard.

12. Known Issues / Technical Debt

Discovered by cross-checking the live Supabase schema (Section 7) against the queries the admin panel actually runs (Section 6). These are functional bugs, not style preferences:

#	Location	Issue
1	<code>admin-dashboard.js</code>	Queries table <code>news_articles</code> (doesn't exist) with columns <code>published/published_date</code> ; the real table is <code>news</code> with <code>is_published/published_date</code> . The News stat card will silently fail to populate.
2	<code>admin-tenders.js</code>	Selects/writes column reference; the real column is <code>reference_number</code> . Reference numbers entered in the admin form are likely not being saved.
3	<code>admin-press-room.js</code>	Uses columns <code>category</code> , <code>published</code> , <code>published_at</code> ; the real table <code>press_releases</code> has neither <code>category</code> nor <code>published</code> nor <code>published_at</code> (it has <code>is_published</code> and <code>published_date</code>). This module's list/filter is very likely broken against the current schema.
4	<code>admin-newsletter.js</code>	Orders by <code>subscribed_at</code> ; the real column is <code>created_at</code> . Sorting will error or silently no-op.

#	Location	Issue
5	Public newsletter opt-in	newsletter_subscribers exists and is fully manageable from the admin panel, but no public-facing signup form was found anywhere in the codebase (footer.js has no email-capture field). Subscribers can currently only be added by an admin manually, or via direct API calls.
6	admin_audit_log	Table exists (presumably intended to log every INSERT/UPDATE/DELETE an admin makes) but nothing in the codebase writes to it yet — 0 rows.
7	site_settings	Key/value settings table exists but no admin page or public page reads/writes it yet — 0 rows.
8	Media uploads	No file-upload UI anywhere; every image/video field is a plain text URL input. Content editors must upload to R2 out-of-band and paste the URL.
9	.env vs. reality	The .env file uses Next.js-style NEXT_PUBLIC_ variables that nothing in this static site reads. Either the site was meant to be rebuilt on Next.js and wasn't, or the file is aspirational documentation. Whichever is true, Section 8.2 (hardcoded values) is the actual source of truth for the deployed site today.

13. Version History

Git commit log (oldest to newest), 12 commits total:

Commit	Message
26f7dc6	first commit, transferred to static web
84216c3	Added more resources
84f2b85	fixed resources
8e0b0a6	fixed resources again
180dfb5	.
657d19b	removed some pages
07a0183	before my final commit
e189c34	cleaned up by separating css and js in separate files
7458d5b	final checks
d4e025e	fixed downloads loading files from db
000da27	removed some pages
43ac21b	fixed sending enquiries (most recent)

Appendix A: Full Repository File Tree

```
.env, .gitignore, _headers
about.html, bulk-orders.html, contact.html, export.html, faqs.html, index.html,
sitemap.html, video-tour.html
about/ careers/ downloads/ events/ gallery/ news/ press-room/ products/
sustainability/ tenders/ resources/
admin/ login.html index.html news.html gallery.html careers.html tenders.html
events.html press-room.html downloads.html inquiries.html newsletter.html
settings.html
css/ main.css admin.css pages/*.css (one per public + admin page)
js/ supabase-client.js common-init.js nav.js footer.js inquiry-modal.js
tailwind-config.js pages/*.js (one per public + admin page)
```

Appendix B: Quick-Reference Credential Sheet

Service	Key / value	Secret
Supabase URL	https://ffsddbdtgoxbqlrncrm.supabase.co	No
Supabase anon key	eyJhbGci...Bb3I (see 8.1/8.2 for full JWT)	Public-safe, but still an app key
Supabase service-role key	eyJhbGci...3JA (see 8.1/8.2 for full JWT)	YES — full DB bypass
Supabase DB password	e3LE5cPyNNr9Mwwy	YES
Supabase publishable key	sb_publishable_BQvGZjG29groWD4mBMKpDg_9GDfwq-O	No (by design)
Cloudflare account ID	4fb1ca4024d02baee6d3b7836495889c	Semi-sensitive
R2 access key ID	53b0a94bc2585c8577c4e3273d8f279e	YES
R2 secret access key	d966077b4cccb70c6973ea0771389869e8fc8481c2e30c53cd98e15db45cb33e	YES
Resend API key	re_placeholder_add_your_key_here (not yet real)	N/A — placeholder
WhatsApp business number	+256 749 293610	No